

Decisiones técnicas destacadas

Esta sección documenta los aspectos del proyecto que se salen de una arquitectura convencional y que requieren justificación explícita.

Por qué Supabase en lugar de un backend tradicional

Supabase es un Backend as a Service (BaaS) que proporciona base de datos PostgreSQL, autenticación, almacenamiento y API REST autogenerada. La decisión de usarlo en lugar de construir un backend propio con Node.js o Spring se basa en tres razones: primero, el proyecto no requiere lógica de negocio server-side compleja que justifique un servidor dedicado; segundo, Supabase ofrece autenticación JWT, Row Level Security y políticas de acceso a nivel de base de datos, lo que cubre los requisitos de seguridad sin código adicional; tercero, permite desarrollar y desplegar más rápidamente en un proyecto acotado en tiempo. Las operaciones que sí requieren servidor (protección de claves privadas, envío de correo, chatbot) se resuelven con funciones serverless en Vercel.

Repository Pattern sobre Supabase

Aunque Supabase podría llamarse directamente desde los componentes, se ha aplicado el patrón Repository para desacoplar la capa de acceso a datos del resto de la aplicación. Cada entidad de negocio tiene una interfaz TypeScript (por ejemplo, `ProductoRepository`) y una implementación concreta (`SupabaseProductoRepository`). Los componentes y páginas interactúan únicamente con la interfaz, nunca con el cliente de Supabase directamente. Esto permite sustituir Supabase por otro proveedor modificando solo las implementaciones, sin tocar la lógica de presentación. Las instancias se crean mediante funciones factory (`createProductoRepository`, `createCompraRepository`, etc.) definidas en `src/database/repositories/index.ts`.

Row Level Security (RLS) en Supabase

Row Level Security es un mecanismo de PostgreSQL que aplica políticas de acceso a nivel de fila directamente en la base de datos, antes de que los datos lleguen a la aplicación. En `LookControl`, cada tabla tiene habilitado RLS con políticas que comparan el `id_peluqueria` del usuario autenticado (obtenido del JWT de Supabase mediante la función auxiliar `get_my_id_peluqueria()`) con el `id_peluqueria` de cada registro. Esto garantiza que un usuario de una peluquería jamás pueda leer ni modificar datos de otra, incluso si la URL de la API fuera accedida directamente. Las funciones SQL auxiliares `get_my_rol()`, `get_my_id_perfil()`, `get_my_id_peluqueria()` y `get_my_permisos()` encapsulan la lectura del contexto del usuario autenticado y son invocadas por las políticas RLS de cada tabla.

Sistema de recuperación de contraseña propio

Supabase incluye recuperación de contraseña nativa, pero requiere redirigir al usuario a una URL del dominio registrado mediante un enlace en el correo. Dado que el proyecto opera sin dominio personalizado estable durante el desarrollo, se implementó un sistema propio

basado en códigos numéricos de 6 dígitos. El flujo es: el usuario solicita el código, el endpoint `/api/password-reset-request` genera el código, lo hashea con HMAC-SHA256 usando una clave secreta (`PASSWORD_RESET_SECRET`) y lo almacena en la tabla `password_reset_codes` junto a su fecha de expiración (15 minutos). El código en texto plano se envía por correo. El endpoint `/api/password-reset-confirm` valida que el hash coincida, que no haya expirado y que no se hayan superado 5 intentos, y actualiza la contraseña usando `SUPABASE_SERVICE_ROLE_KEY`, que tiene permisos de administrador sobre la base de datos y solo se usa en servidor.

Funciones serverless en Vercel

Los archivos en `/api/` (`contact.js`, `password-reset-request.js`, `password-reset-confirm.js` y `groq.js`) son funciones serverless que Vercel ejecuta como endpoints HTTP independientes. Su propósito es proteger claves que no pueden exponerse en el frontend: `SUPABASE_SERVICE_ROLE_KEY`, `GROQ_API_KEY`, `SMTP_PASS` y `RESEND_API_KEY`. El bundle de React que llega al navegador no contiene estas claves en ningún momento. Las llamadas del frontend van a `/api/*`, Vercel las enruta a la función correspondiente, que ejecuta la operación con la clave privada y devuelve solo el resultado. En desarrollo local, Vite emula este comportamiento mediante su plugin de funciones serverless configurado en `vite.config.ts`.

Chatbot con Groq y Llama 3.3

El chatbot usa Groq como proveedor de inferencia con el modelo Llama 3.3 70B Versatile. Groq se eligió por su tier gratuito con límites suficientes para un proyecto académico y por su latencia extremadamente baja gracias al hardware LPU (Language Processing Unit) propio. Llama 3.3 70B es un modelo open-source de Meta que ofrece calidad comparable a modelos comerciales para tareas de soporte y Q&A. La llamada no se hace directamente desde el frontend para no exponer la `GROQ_API_KEY`; en su lugar, el frontend llama a `/api/groq`, que actúa como proxy hacia la API de Groq. El historial de conversación se mantiene en estado local del componente (`useState`) y se envía completo en cada petición para que el modelo tenga contexto de la conversación anterior.

Soft delete en lugar de eliminación física

Productos, servicios y proveedores no se eliminan físicamente de la base de datos cuando el usuario pulsa “eliminar”. En su lugar, se marca el campo activo como false. Esto se denomina soft delete o borrado lógico. La razón es que estas entidades tienen referencias en otras tablas: un producto eliminado puede estar en líneas de compra o movimientos de stock históricos; si se borrara físicamente, se perdería la trazabilidad y se violarían las restricciones de clave foránea. Los proveedores tienen además una opción de eliminación física (`hardDelete`) disponible solo cuando ya están inactivos y no tienen compras asociadas. Las consultas filtran siempre por `activo = true` para mostrar solo registros vigentes.

Gestión de estado global con Zustand

Zustand se usó en lugar de Redux o Context API por su API mínima y sin boilerplate. El único store global es `useAuthStore`, que almacena el perfil del usuario, el estado de

autenticación y las acciones de login, logout e `initSession`. Se configura con el middleware `persist` de `Zustand`, que serializa el store en `localStorage` bajo la clave `lookcontrol-auth-v1` para que la sesión sobreviva a recargas de página. Al cargar la aplicación, `initSession()` verifica con Supabase que el token siga siendo válido antes de dar acceso a las rutas privadas. El resto del estado (datos de productos, compras, etc.) es local a cada página y no se eleva al store global, siguiendo el principio de mínima superficie de estado compartido.

Sistema de envío de correo dual (SMTP y Resend)

Los endpoints de envío de correo soportan dos proveedores de forma alternativa: SMTP estándar (configurado con `Nodemailer`, compatible con Gmail u otros servidores) y Resend (servicio transaccional con API REST). Si las variables de entorno de Resend están definidas, se usa Resend; en caso contrario, se usa SMTP. Esta dualidad permite que el proyecto funcione tanto en entornos de desarrollo local (con una cuenta SMTP de Gmail) como en producción (con Resend, que ofrece mejor entregabilidad y un tier gratuito). No es redundancia sino una estrategia de fallback por entorno.